Подготовка к защите лабораторной работы

Дутов Р. В. • C++17 • Варианты 1 и 2

Пособие составлено по текущему коду prdxq1/codex, коммит c767f7b. Ссылки вида «src/Filters.cpp:75» обозначают файл и номер строки. Это пояснение к реализации, а не замена отчёта.

Короткий рассказ о работе — примерно на минуту

«Я реализовал хранение и обработку PNG-изображений. Класс Image отвечает за размеры, каналы, память и отдельные пиксели. Массив принадлежит unique_ptr, поэтому освобождается автоматически. При копировании Image создаётся собственный массив. Фильтры находятся отдельно в пространстве Filters и не меняют исходный объект. В первом варианте использованы Собел, Робертс и Лаплас, во втором — прямоугольное и гауссово усреднение. Для проверки программа создаёт треугольник, ромб, эллипс и крест, добавляет гауссов шум и сохраняет результаты. На границах берётся ближайший допустимый пиксель: это делает clampCoord».

Как пользоваться пособием

Страницы	Что разобрано
2–3	Карта проекта; пиксели, каналы и умный указатель
4–5	Конструкторы, глубокое копирование, присваивание самому себе
6	Чтение PNG, запись, getColor/setColor
7–8	Индексы, clamp и все 9 обращений к угловому пикселю
9–14	Каждый фильтр: код, маски и численный расчёт
15	Первый пиксель настоящего изображения и фактические результаты
16–17	Вопросы преподавателя, ограничения и план демонстрации

Главное обозначение: I(x, y) — яркость; x — столбец, y — строка; отсчёт в C++ начинается с нуля. Первый пиксель — (0, 0). Если преподаватель называет его [1, 1] при отсчёте с единицы, это тот же верхний левый угол.

1. Где что находится в проекте

Файл / строки	Назначение
include/Image.h:11–37	Объявление Image. Размеры, число каналов, размер массива, unique_ptr.
src/Image.cpp:12–38	Пустой конструктор, создание растра, загрузка из файла.
src/Image.cpp:40–65	Конструктор копирования и копирующее присваивание.
src/Image.cpp:67–128	isRead, isWrite, setColor, getColor.
src/Filters.cpp:11–43	clampCoord, clampByte, чтение канала, перевод в полутоновое изображение.
src/Filters.cpp:50–136	applySobel, applyRoberts, applyLaplacian.
src/Filters.cpp:143–220	applyBoxBlur, applyGaussianBlur.
src/Source.cpp:12–39	Создание фигур и добавление шума.
src/Source.cpp:44–78	MSE/PSNR, main, вызовы фильтров и сохранение файлов.
tests/tests_image.cpp; tests/tests_filters.cpp	Проверки класса и математических свойств фильтров.

Что происходит после запуска

main создаёт каталоги → generateScene строит чистую сцену → addNoise делает её независимую копию с шумом → сохраняются оба входа → по очереди вызываются фильтры → сохраняются PNG и metrics.csv.

Для контуров также сохраняется обработка чистого эталона с префиксом clean_. Это помогает отличить настоящую границу фигуры от отклика на шум.

Почему фильтры не являются методами Image

По заданию нельзя расширять интерфейс Image. Поэтому функции обработки находятся в namespace Filters. Они получают const Image&: изображение передаётся без копирования аргумента и не изменяется через эту ссылку. Для результата создаётся отдельный dst.

2. Память, пиксели и unique_ptr

```
size_t m_size{0};
std::unique_ptr<uint8_t[]> m_data;
// include/Image.h:35–36
```

m_data — умный указатель с единственным владельцем массива байтов. Скобки [] в типе означают владение массивом: при уничтожении используется delete[], а не delete для одного элемента. Сам указатель нельзя копировать обычным присваиванием; поэтому Image реализует собственное глубокое копирование.

```
m_size = static_cast<size_t>(m_width) * m_height * m_nChannels;
m_data = std::make_unique<uint8_t[]>(m_size);
// src/Image.cpp:17–18
```

make_unique выделяет массив, которым сразу владеет m_data. Для 256×256 с одним каналом это 65 536 байтов пикселей; для RGB — 196 608. size_t используется для размеров и индексов. Поля объекта и служебные данные в эти числа не включены.

Как двумерный растр помещается в одномерный массив

```
index = (static_cast<size_t>(y) * m_width + x) * m_nChannels;
```

Сначала пропускаются y полных строк, затем x пикселей текущей строки; умножение на число каналов переводит номер пикселя в смещение по байтам. RGB записывается в index, index+1 и index+2.

Пример	Смещение
Пиксель (0,0), RGB	(0×W+0)×3 = 0
Пиксель (1,1), W=256, RGB	(1×256+1)×3 = 771
Тот же пиксель, один канал	(1×256+1)×1 = 257

Кто освобождает память

В Image.h:19 написано ~Image() = default. При разрушении Image автоматически разрушается m_data и освобождает массив — явный delete не нужен. m_data.reset() освобождает старый массив и делает указатель пустым; присваивание нового unique_ptr также автоматически освобождает прежний массив. m_data.get() лишь даёт сырой адрес для memcpy или библиотеки, владение при этом не передаётся.

Что сказать на защите. «Умный указатель находится в закрытом поле m_data. Он отвечает за время жизни массива; доступ к байтам организуют методы Image».

3. Конструкторы и глубокое копирование

Выражение	Что вызывается
Image a;	Пустой конструктор, Image.cpp:12. Поля уже имеют нулевые значения.
Image a(256,256,1);	Конструктор по размерам, строки 14–32.
Image a("input.png");	Конструктор по пути, строки 34–38.
Image b(a); или Image b = a;	Конструктор копирования, строки 40–47. b только создаётся.

Конструктор по размерам проверяет положительные W, H и 1–4 канала. При неверных параметрах получается пустой объект. При допустимых выделяется массив, яркости обнуляются, а альфа-канал для двух и четырёх каналов заполняется 255. Конструктор по пути вызывает isRead; если чтение не удалось, бросает runtime_error.

Копирующий конструктор — построчно

```
Image::Image(const Image& other)
  : m_width(other.m_width), m_height(other.m_height),
    m_nChannels(other.m_nChannels), m_size(other.m_size) {
    if (other.m_data && m_size > 0) {
        m_data = std::make_unique<uint8_t[]>(m_size);
        std::memcpy(m_data.get(), other.m_data.get(), m_size);
    }
}
```

1. Список инициализации переносит размеры и число каналов. 2. Проверяется наличие данных. 3. Выделяется новый массив. 4. memcpy копирует m_size байтов. Копируются именно данные, а не адрес указателя. Для массива uint8_t побайтовое копирование корректно.

Если other пуст, условие не выполняется, m_data остаётся пустым. Копия тоже представляет пустое изображение.

```
Image a(2, 2, 1);
a.setColor(0, 0, 100, 0, 0);
Image b(a);                 // новый массив, значение 100
a.setColor(0, 0, 200, 0, 0); // у b по-прежнему 100
```

Что сказать на защите. «В addNoise строка Image result(clean) вызывает именно этот конструктор. Поэтому добавление шума в result не портит чистый эталон».

4. operator= и защита от самоприсваивания

Конструктор копирования создаёт новый объект. Копирующее присваивание заменяет данные уже существующего объекта. Пример: Image b; b = a; — здесь вызывается operator=, а не конструктор.

```
Image& Image::operator=(const Image& other) {
    if (this == &other) return *this;
    m_width = other.m_width;
    m_height = other.m_height;
    m_nChannels = other.m_nChannels;
    m_size = other.m_size;
    if (other.m_data && m_size > 0) {
        m_data = std::make_unique<uint8_t[]>(m_size);
        std::memcpy(m_data.get(), other.m_data.get(), m_size);
    } else {
        m_data.reset();
    }
    return *this;
}
```

Что означает if (this == &other;)

this — адрес объекта слева от присваивания; &other — адрес источника. При a = a это один и тот же адрес, поэтому функция сразу возвращает текущий объект. Сравниваются адреса, а не размеры или содержимое картинок.

Без этой проверки при a = a строка с make_unique заменила бы m_data самого источника, одновременно уничтожив прежний массив. Последующий memcpy уже не восстановил бы исходные пиксели. Ранний return сохраняет их и исключает ненужное выделение памяти.

Обычное присваивание и пустой источник

При b = a поля переносятся из a, для b выделяется новый массив, старый массив b автоматически освобождается, затем копируются байты a. Если a пуст, reset очищает данные b. return *this возвращает ссылку на получатель, поэтому возможна цепочка a = b = c.

Точная граница гарантии: реализация не даёт строгой гарантии сохранения состояния при bad_alloc: размеры меняются до успешного выделения памяти. В обычном успешном сценарии копирование глубокое. Утверждать, что код уже использует copy-and-swap, нельзя — такого приёма здесь нет.

5. Чтение PNG и работа с каналами

isRead — Image.cpp:67–80

stbi_load загружает файл и сообщает ширину, высоту и число каналов. Последний аргумент равен 0: исходное число каналов сохраняется. Если возвращён nullptr, функция возвращает false. При успехе создаётся собственный массив, байты копируются из raw, затем raw освобождается через stbi_image_free.

raw — временный сырой указатель из сторонней библиотеки, не unique_ptr. Его нельзя освобождать обычным delete[]. Результатом Image владеет уже m_data. При ошибке чтения до загрузки существующее изображение не заменяется.

isWrite — Image.cpp:83–88

Пустой растр не сохраняется: возвращается false. stride = width × nChannels — длина одной строки в байтах. В stbi_write_png передаются размеры, число каналов, адрес данных и stride. В main используется обёртка save, которая превращает false в исключение с понятным сообщением.

getColor / setColor — Image.cpp:91–128

Каналы	Запись setColor	Чтение getColor
1, grayscale	Сохраняется r.	В r записывается яркость; g и b не меняются.
2, gray + alpha	Яркость r и alpha=255.	Возвращается яркость в r.
3, RGB	Сохраняются r, g, b.	Читаются три канала.
4, RGBA	RGB сохраняются, alpha=255.	Читается RGB; alpha наружу не выдаётся.

Проверки !m_data и выхода за границы выполняют ранний return. При недопустимом чтении выходные r/g/b остаются прежними. Поэтому в фильтрах они заранее инициализируются нулями.

Два разных механизма: getColor сам ничего не прижимает к границе. Это делает clampCoord до вызова getColor. setColor сам не ограничивает яркость до 255: он приводит int к uint8_t; поэтому диапазон проверяется на уровне алгоритма.

6. Координаты и два разных clamp

```
inline int clampCoord(int val, int maxVal) {
    return std::clamp(val, 0, maxVal - 1);
}
inline int clampByte(int val) {
    return std::clamp(val, 0, 255);
}
```

std::clamp(v, lo, hi) возвращает lo, если v ниже lo; hi, если v выше hi; иначе само v. Это стандартная функция C++17 из <algorithm>. clampCoord и clampByte — разные обёртки вокруг неё.

Вызов	Ответ	Зачем
clampCoord(-1,256)	0	Не читать левее или выше растра.
clampCoord(1,256)	1	Допустимая координата остаётся прежней.
clampCoord(256,256)	255	Не читать правее или ниже растра.
clampByte(-20)	0	Ограничить отрицательную яркость.
clampByte(300)	255	Убрать переполнение яркости.

Где вызывается clampCoord

Для контуров цепочка такая: applySobel / applyRoberts / applyLaplacian → getPixelChannel → clampCoord для x и y → Image::getColor. См. Filters.cpp:20–28. Для Box Blur и Gaussian Blur clampCoord вызывается прямо перед getColor, строки 159 и 207.

Какой способ продолжения за границу выбран

Крайний допустимый пиксель повторяется столько раз, сколько обращений в него попало. Это репликация границы. Нет зеркального отражения, перехода на другой край и заполнения нулями. Поэтому в углу окно 3×3 всё равно содержит девять слагаемых, хотя реальных разных пикселей всего четыре.

Для изображения 1×1 любая запрошенная координата прижимается к 0. Для пустого изображения в фильтрах не выполняются циклы по пикселям; clampCoord с maxVal=0 вызывать нельзя: интервал [0,−1] недопустим.

Что сказать на защите. «clampCoord защищает координату; clampByte ограничивает значение яркости. Одно нельзя заменить другим».

7. Первый пиксель: все обращения окна 3×3

Возьмём небольшой учебный растр, чтобы арифметика была видна. Он используется только в объяснении, а не вместо ваших тестовых изображений. Нумерация с нуля; строки матрицы — y, столбцы — x.

```
Исходные яркости I(x,y):
        x=0  x=1  x=2
y=0      10   20   30
y=1      40   50   60
y=2      70   80   90
```

Пусть x=0, y=0. Смещения kx и ky перебираются от −1 до 1. До clamp запрашивается (x+kx, y+ky). Индекс коэффициента — [ky+1][kx+1]: прибавление единицы переводит смещения −1,0,1 в индексы 0,1,2.

ky, kx	До clamp: (x,y)	После clamp	Пиксель	Индекс маски
−1, −1	(−1,−1)	(0,0)	10	[0][0]
−1, 0	(0,−1)	(0,0)	10	[0][1]
−1, 1	(1,−1)	(1,0)	20	[0][2]
0, −1	(−1,0)	(0,0)	10	[1][0]
0, 0	(0,0)	(0,0)	10	[1][1]
0, 1	(1,0)	(1,0)	20	[1][2]
1, −1	(−1,1)	(0,1)	40	[2][0]
1, 0	(0,1)	(0,1)	40	[2][1]
1, 1	(1,1)	(1,1)	50	[2][2]

```
Фактическое окно вокруг (0,0):
10  10  20
10  10  20
40  40  50
```

Где читается пиксель изображения (1,1)? При kx=1 и ky=1. Для Собела и Лапласа это последний шаг внутренних циклов. У него индекс маски [2][2], а не [1][1]. Центральный элемент маски [1][1] относится к текущему пикселю (0,0).

8. Собел: что именно вычисляют циклы

Код: src/Filters.cpp:50–85. Сначала создаётся grayscale-копия, затем пустой одноканальный dst. Для каждого x,y обнуляются sumX и sumY. Внутренние циклы умножают девять яркостей на два набора коэффициентов.

```
Gx = [-1  0  1]       Gy = [-1 -2 -1]
     [-2  0  2]            [ 0  0  0]
     [-1  0  1]            [ 1  2  1]

int pixel = getPixelChannel(gray, x+kx, y+ky, 0);
sumX += pixel * Gx[ky+1][kx+1];
sumY += pixel * Gy[ky+1][kx+1];
```

Подстановка окна первого пикселя со страницы 8

```
sumX = -10 + 0 + 20
       -20 + 0 + 40
       -40 + 0 + 50 = 40

sumY = -10 - 20 - 20
         0 +  0 +  0
        40 + 80 + 50 = 120

magnitude = sqrt(40*40 + 120*120) = 126.491...
static_cast<int>(magnitude) = 126
clampByte(126) = 126
```

Результат записывается в dst(0,0). Исходный gray не меняется: следующий пиксель считается по исходным значениям, а не по уже обработанным соседям.

Тонкость с [1][1]

У обеих масок центр [1][1] равен нулю. Центр всё равно читается циклом, но его вклад в суммы нулевой. А соседний пиксель изображения (1,1)=50 умножается на нижний правый коэффициент [2][2]: +1 в Gx и +1 в Gy.

Собел оценивает первые производные. Его коэффициенты дают сглаживание поперёк направления разности, но не устраняют шум полностью. Маски не делятся на 8: большие амплитуды обрезаются до 255, а не нормируются по максимуму всего изображения.

9. Робертс: четыре пикселя без вложенной маски

Код: src/Filters.cpp:88–108. Здесь нет циклов по ядру, потому что компоненты 2×2 выписаны явно. Перед обработкой вход также переводится в grayscale.

```
int p00 = getPixelChannel(gray, x,     y,     0);
int p11 = getPixelChannel(gray, x + 1, y + 1, 0);
int p01 = getPixelChannel(gray, x + 1, y,     0);
int p10 = getPixelChannel(gray, x,     y + 1, 0);
double gx = p00 - p11;
double gy = p01 - p10;
```

Имена p01 и p10 согласованы с записью «строка, столбец»: p01 — строка 0, столбец 1, то есть справа; p10 — ниже. Не нужно трактовать суффикс как пару (x,y). В API getColor порядок аргументов другой: именно x, затем y.

```
Эквивалентные маски:
Rx = [1  0]          Ry = [ 0  1]
     [0 -1]               [-1  0]
```

Первый пиксель учебного растра

```
p00=10; p11=50; p01=20; p10=40
gx = 10-50 = -40
gy = 20-40 = -20
sqrt(gx*gx+gy*gy) = sqrt(2000) = 44.721...
После приведения к int и clampByte: 44
```

Обращение к изображению (1,1) находится прямо в строке 97: при x=y=0 выражение x+1,y+1 становится 1,1. Для обычного растра это допустимый пиксель, поэтому clampCoord его не меняет.

Что происходит на правом и нижнем краях

У правого края x+1 прижимается к W−1, у нижнего y+1 — к H−1. В самом нижнем правом углу все четыре чтения возвращают один пиксель. Обе разности равны нулю, значит результат там всегда 0. У остальных граничных позиций разности могут быть ненулевыми.

Что сказать на защите. «Робертс сравнивает яркости по двум диагоналям блока 2×2, затем берёт длину вектора разностей. Небольшое окно даёт тонкие линии, но чувствительно к шуму».

10. Лаплас: вторые разности и модуль

Код: src/Filters.cpp:111–136. Схема обхода совпадает с Собелом, но маска одна, а вместо длины двух компонент берётся модуль одной суммы.

```
kernel = [0  1  0]
         [1 -4  1]
         [0  1  0]

int pixel = getPixelChannel(gray, x+kx, y+ky, 0);
sum += pixel * kernel[ky+1][kx+1];
int val = clampByte(std::abs(sum));
```

Внутри изображения это сумма четырёх соседей минус четыре центральных значения: $L = I(x-1,y)+I(x+1,y)+I(x,y-1)+I(x,y+1)-4I(x,y)$. Так складываются конечные разности второго порядка по x и по y.

Первый пиксель учебного растра

```
Окно после clamp:        Вклад:
10 10 20                  0 +10 + 0
10 10 20                 10 -40 +20
40 40 50                  0 +40 + 0

sum = 40; abs(40)=40; clampByte(40)=40
```

В углу $I(-1,0)$ и $I(0,-1)$ заменяются на $I(0,0)$. Поэтому формула упрощается до $I(1,0)+I(0,1)-2I(0,0)$. Для примера: $20+40-2\times10=40$.

Читается ли здесь изображение (1,1)?

Да. При kx=ky=1 выполняется getPixelChannel(gray,1,1,0). Но коэффициент kernel[2][2] равен 0: вклад значения 50 в сумму нулевой. Цикл обходит все девять ячеек, хотя ненулевых коэффициентов в маске пять.

Почему abs до clamp

При отрицательном отклике, например −80, модуль даёт 80. Если сначала ограничить его диапазоном 0–255, он превратится в 0 и одна полярность границы пропадёт. В данной визуализации знак намеренно теряется; это не алгоритм поиска нулевых пересечений.

Что сказать на защите. «Собел и Робертс оценивают первую производную, Лаплас — вторую. Постоянная яркость у всех трёх должна давать нулевой отклик».

11. Прямоугольное усреднение: 3×3 и 5×5

Код: src/Filters.cpp:143–169. Размер проверяется: допустимо нечётное kernelSize от 1 до 255. radius=kernelSize/2; count=kernelSize×kernelSize. Для 3×3 это radius=1 и count=9; для 5×5 — 2 и 25.

```
for (int ky=-radius; ky<=radius; ++ky) {
    for (int kx=-radius; kx<=radius; ++kx) {
        int r=0, g=0, b=0;
        src.getColor(clampCoord(x+kx,w),
                     clampCoord(y+ky,h), r,g,b);
        sumR += r; sumG += g; sumB += b;
    }
}
dst.setColor(x,y,sumR/count,sumG/count,sumB/count);
```

Первый пиксель учебного растра, окно 3×3

```
После clamp:
10  10  20
10  10  20
40  40  50

sumR = 4*10 + 2*20 + 2*40 + 50 = 210
count = 9
210/9 = 23   // деление целых, дробная часть отброшена
```

Пиксель изображения (1,1) попадает в сумму при kx=1,ky=1 один раз. Центральный (0,0) встречается четыре раза: три обращения с отрицательными координатами прижаты к нему плюс само центральное обращение. Делить на четыре различных пикселя нельзя: это была бы другая граничная обработка.

Как выглядит 5×5 в том же углу

```
Смещения по каждой оси: -2 -1  0  1  2
После clamp:             0  0  0  1  2

10 10 10 20 30
10 10 10 20 30
10 10 10 20 30
40 40 40 50 60
70 70 70 80 90
```

Сумма 850; 850/25=34. Более широкое окно объединяет больше отсчётов: на ровном фоне шум уменьшается, но граница фигуры размазывается. Для grayscale getColor меняет только r; сумма по r даёт нужную яркость, а setColor в одноканальном dst использует только r.

12. Гаусс: построение и нормировка ядра

Код: src/Filters.cpp:172–198. Размер окна проверяется так же, как у Box Blur. sigma должно быть конечным числом в диапазоне 10^{-6}...10^{6}. В main используется окно 5×5 и sigma=1,5; шум имеет другое σ=25 — эти два параметра нельзя путать.

```
double s = 2.0 * sigma * sigma;
double r = x*x + y*y;
double val = std::exp(-r/s) / (3.14159265358979323846*s);
kernel[y+radius][x+radius] = val;
sumWeights += val;
// После заполнения:
kernel[i][j] /= sumWeights;
```

На этапе создания ядра x и y — локальные смещения от его центра, а не координаты пикселя изображения. r — квадрат расстояния. Значение val максимально в центре и убывает к краям. vector<vector<double>> хранит коэффициенты и сам управляет своей памятью.

Почему недостаточно подставить формулу распределения

Непрерывная функция интегрируется в единицу на бесконечной плоскости. Здесь берётся лишь конечный набор отсчётов. Поэтому каждый вес делится на их фактическую сумму: после этого сумма элементов дискретного ядра равна единице с точностью double. Постоянный фон не должен темнеть или светлеть.

Фактическая маска 5×5, sigma=1,5

```
0.014419 0.028084 0.035073 0.028084 0.014419
0.028084 0.054700 0.068312 0.054700 0.028084
0.035073 0.068312 0.085312 0.068312 0.035073
0.028084 0.054700 0.068312 0.054700 0.028084
0.014419 0.028084 0.035073 0.028084 0.014419
```

Числа здесь округлены для чтения; в программе используются неокруглённые double. Центральный элемент окна 5×5 — [2][2], а не [1][1]. Делить результат ещё на 25 нельзя: веса уже нормированы.

Что сказать на защите. «В равномерном усреднении каждый отсчёт имеет вес $1/k^2$; у Гаусса близкие к центру имеют больший вес. Это всё ещё линейное усреднение до округления».

13. Гаусс: первый пиксель и доступ к (1,1)

Код применения: Filters.cpp:200–217. Окно обходится, как у Box Blur, но каждая яркость умножается на свой вес. Суммы accR, accG, accB имеют тип double.

```
src.getColor(clampCoord(x+kx,w),
             clampCoord(y+ky,h), r,g,b);
double weight = kernel[ky+radius][kx+radius];
accR += r*weight;
// Для неотрицательного результата:
int value = clampByte(static_cast<int>(accR+0.5));
```

Учебное окно 3×3, sigma=1

```
Нормированное ядро:         Окно y (0,0):
0.075114 0.123841 0.075114    10 10 20
0.123841 0.204180 0.123841    10 10 20
0.075114 0.123841 0.075114    40 40 50
```

Учитывая повторяющиеся пиксели, accR ≈ 10×0,526976 + 20×0,198955 + 40×0,198955 + 50×0,075114 = 20,962745. Прибавление 0,5 и приведение к int дают 21. Пиксель (1,1)=50 умножается на kernel[2][2].

Окно 5×5, sigma=1,5 — именно параметры main

Смещения по каждой оси после clamp переходят в 0,0,0,1,2. Значение I(0,0)=10 повторяется девять раз, I(1,0) и I(0,1) — по три раза. Каждое повторение берётся со своим весом. Маска не обрезается и не пересчитывается заново для угла.

Для учебной матрицы со страницы 8 сумма по 25 весам равна 28,961501, итоговая яркость — 29. Здесь пиксель изображения (1,1) читается при kx=1,ky=1 и умножается на kernel[3][3] ≈ 0,054700. Сдвиг индекса равен radius=2.

Объект при x=y=0	Окно 3×3	Окно 5×5
Центр маски для I(0,0)	[1][1]	[2][2]
Вес для I(1,1)	[2][2]	[3][3]

Проверка чисел: значения 126, 44, 40, 23, 21 и 29 на страницах 9–14 сверены непосредственным вызовом ваших C++-функций на одной и той же учебной матрице.

14. Первый пиксель настоящего изображения

Для файла test_input/input.png из коммита c767f7b верхний левый участок выглядит так. Значения приведены после генерации шума и ограничения яркости; код повторно прочитал именно PNG.

```
181 208 207
156 186 200
225 201 197

Окно 3x3 после clamp вокруг (0,0):
181 181 208
181 181 208
156 156 186
```

Метод	Расчёт или результат для (0,0)
Собел	sumX=111, sumY=−97; sqrt(21730) → 147.
Робертс	gx=181−186=−5; gy=208−156=52; sqrt(2729) → 52.
Лаплас	208+156−2×181 = 2.
Box Blur 3×3	(4×181+2×208+2×156+186)/9 = 182.
Гаусс 3×3, σ=1	Нормированная взвешенная сумма → 182.
Гаусс 5×5, σ=1,5	По полному окну 5×5 → 188. С clamp достаточно показанных 3×3, но складываются 25 вкладов.

Числа подтверждены запуском ваших фильтров. При другой стандартной библиотеке C++ распределение normal_distribution с тем же seed может дать другой шум; геометрия, маски и правило clamp останутся теми же.

Что видно по метрикам всей новой сцены

Изображение	MSE	PSNR, дБ	MSE ровного фона
С шумом	614,546	20,2453	620,891
Box 3×3	119,295	27,3646	74,7996
Box 5×5	113,949	27,5637	34,8829
Гаусс 5×5	96,6393	28,2793	38,2121

Общая ошибка минимальна у Гаусса, а на выбранном ровном фоне — у Box 5×5. Это разные критерии. PSNR считается как $10 \cdot \log_{10}(255^2/\mathrm{MSE})$; MSE — средний квадрат разности с чистым эталоном.

15. Вопросы, на которых легко запутаться

«Чем конструктор копирования отличается от operator=?»

Первый создаёт новый Image; второй меняет уже существующий. Image b=a; — создание через копирующий конструктор, несмотря на знак =. b=a; после объявления b — присваивание. В обоих случаях пиксели копируются в независимый массив.

«Почему для контуров сначала grayscale?»

Операторы вычисляют перепад одной скалярной величины — яркости. Для RGB применяется 0,299R+0,587G+0,114B; для grayscale берётся r. Число каналов результата контуров равно одному.

«Это свёртка или корреляция?»

В коде маска прикладывается как записана, без разворота на 180°: строго это корреляция. Для симметричных ядер Лапласа, Box и Гаусса отличий от свёртки нет. Для Собела и Робертса разворот меняет знаки компонент, но не длину градиента, которую мы визуализируем.

«Почему нельзя вычислять прямо в исходном изображении?»

Тогда следующие позиции использовали бы уже изменённые соседние значения и результат зависел бы от порядка обхода. Здесь чтение идёт из src/gray, запись — в отдельный dst.

«Чем обработка пикселя (1,1) отличается от (0,0)?»

Если считать с нуля, для центра (1,1) и окна 3×3 координаты от 0 до 2 допустимы: clamp их не меняет, окно равно исходной таблице со страницы 8. Для окна 5×5 даже при центре (1,1) ещё есть отрицательные координаты, поэтому clamp нужен. Для проверки: Собел на центре учебной таблицы даёт sumX=80, sumY=240, яркость 252; Лаплас даёт 0.

«Почему Gaussian Blur не устраняет весь шум?»

Усреднение подавляет случайные высокочастотные колебания, но в тех же частотах содержатся тонкие детали и резкие границы. Чем сильнее сглаживание, тем заметнее размытие. Метод не восстанавливает неизвестный идеальный пиксель в точности.

«Какова вычислительная сложность?»

Контурные маски фиксированы, поэтому O(W·H). Наивные усреднения имеют O(W·H·k^2). Здесь гауссово ядро применяется именно двумерно, не разложено на два одномерных прохода. Дополнительная память ядра — O(k^2), выходного изображения — O(W·H·C).

16. Что демонстрировать и чего не обещать

Практическая последовательность защиты

1. Открыть Source.cpp:58–70 и показать весь путь от генерации сцены до сохранения обработанных PNG. Объяснить σ шума=25 и σ сглаживания=1,5.

2. Открыть Image.h:36, затем Image.cpp:40 и 49. Назвать unique_ptr, новый массив при копировании, memcpy и проверку this==&other.

3. Открыть Filters.cpp:11–22. На примере (−1,−1) объяснить получение (0,0). Затем по таблице со страницы 8 пройти первое окно.

4. Показать Собел:75–82, Робертс:96–105, Лаплас:128–133. Объяснить, где пиксель (1,1) используется с ненулевым весом, а где умножается на ноль.

5. Показать Box:159–166 и Гаусс:185–217. У первого сумма делится на число элементов, у второго используются нормированные веса и округление.

```
make -j2
make test
make run
```

Текущий результат тестов: 16 случаев и 547 утверждений, все проходят. Есть проверки постоянного изображения, импульса, симметрии Гаусса, глубокого копирования и неправильных параметров. На постоянном изображении контуры нулевые, а усреднение сохраняет яркость.

Ограничения фактической реализации

• Альфа при setColor выставляется в 255; произвольная прозрачность не фильтруется и не сохраняется при такой перезаписи. Двухканальное чтение возвращает только яркость в r.

• Сырые raw-данные stbi_load освобождаются вручную. Если выделение m_data после загрузки бросит исключение, raw в этой ветке не защищён RAII. Для полной устойчивости здесь нужен отдельный guard с stbi_image_free; в текущем коде его нет.

• Присваивание и загрузка не обеспечивают строгую гарантию сохранения старого состояния при исключении выделения памяти. Экстремально большие размеры не проверяются на все переполнения произведений.

• Специальные move-конструктор и move-присваивание не объявлены; собственные копирующие операции подавляют автоматическое создание соответствующих move-операций. Возврат Image по значению может использовать устранение копирования; NRVO для именованного dst не гарантировано.

Что сказать на защите. «Я могу показать конкретные строки, проверить численный пример и объяснить ограничения. Работа алгоритма и поведение на границе определены кодом, а не только изображением в отчёте».